## List of Experiments

**Total No of Labs: 14**

| S. No. | Programs List | No. of Labs |
|---|---|---|
| **CAD Experiments** | | |
| 1 | Study of basic graphics functions defined in “graphics.h”. | 1 |
| 2 | To implement Digital differential analyzer (DDA) Algorithm for drawing a line segment between two given end points A (x1, y1) and B(x2, y2). | 1 |
| 3 | To implement Bresenham’s line drawing algorithm for drawing a line segment between two given endpoints A (x1, y2) and B(x2, y2). | 1 |
| 4 | To implement Bresenham’s Cicrcle drawing algorithm for drawing a circle with a given center of circle P(x, y) and radius r. | 1 |
| 5 | To apply the basic 2D transformations such as translation, Scaling, Rotation, shearing and reflection for a given 2D object. | 2 |
| 6 | To apply the basic transformations such as translation, Scaling, Rotation for a given 3D object. | 2 |
| 7 | To generate a smooth curve by using Bezier curve technique for a given set of control points. | 1 |
| 8 | To implement Cohen–Sutherland 2D clipping and window–viewport mapping | 1 |
| **Product Design** | | |
| 9 | Introduction to CAD software’s. | 1 |
| 10 | Generation of any one simple solid model using any CAD software. | 1 |
| 11 | Generation of any one assembly of mechanical part using any CAD software. | 1 |
| 12 | Design and develop the complete lifecycle for any product. | 1 |

# Experiment - 3

**Aim:** To implement Bresenham's line drawing algorithm for drawing a line segment between two given endpoints A (x1, y2) and B(x2, y2).

**Description:**
The disadvantages associated with DDA algorithm such as real number coordinate calculations and rounding off operation of real numbers are eliminated in Bresenham's algorithm. Bresenham developed an accurate and efficient method of raster generation algorithms to display lines, circles, ellipses and other curves incorporating only incremental integer calculations.

The scan conversion of a line with positive slope less than unity is considered. Let the starting point of the line is $(x_A, y_A)$. Suppose we have calculated the current pixel location for the line as $(x_k, y_k)$, as shown in Fig. 1. Now, this is decided that which pixel should be selected for display at the sampling column $x_{k+1}$. Alternatively, we have two options for the selection of new pixel location, $(x_{k+1}, y_k)$ or $(x_{k+1}, y_{k+1})$. Let, at the sampling column $x_{k+1}$, the vertical separations between the pixels $(x_{k+1}, y_k)$ and $(x_{k+1}, y_{k+1})$, from the mathematically actual line path be $d_1$ and $d_2$, respectively (Fig. 2). At the sampling column $x_{k+1}$ , $y$ coordinate is given as

$$y = m.x_{k+1} + c \tag{1}$$

Moreover, vertical separations $d_1 = y - y_k = (m.x_{k+1} + c) - y_k$ (2)

$$d_2 = y_{k+1} - y = y_{k+1} - (m.x_{k+1} + c) \tag{3}$$

Therefore,

$$\begin{aligned} d_1 - d_2 &= \{(m.x_{k+1} + c) - y_k\} - \{y_{k+1} - (m.x_{k+1} + c)\} \\ &= 2.m.x_{k+1} - y_k - y_{k+1} + 2.c \\ &= 2.m.x_{k+1} - 2.y_k + 2.c - 1 \qquad \text{(because } y_{k+1} = y_k + 1\text{)} \end{aligned} \tag{4}$$

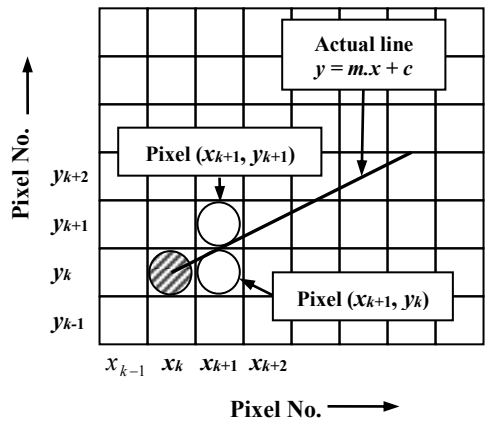


Figure 1: Choice of pixel in new location at sampling column $x_{k+1}$

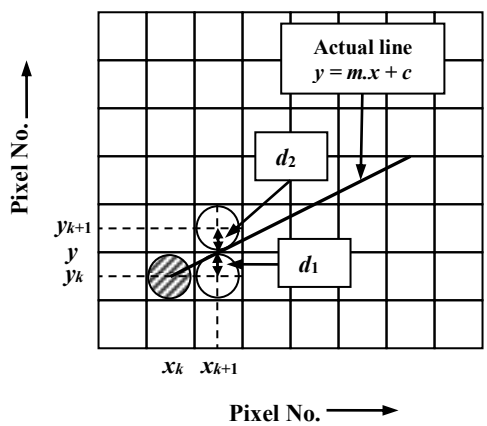


**Figure 2:** Vertical separations between pixel positions $y_{k+1}$ and $y_k$

A decision parameter $P_k$ for the $k^{\text{th}}$ step in line drawing algorithm calculated by rearranging the terms in eqn. (4) so that it incorporates only integer calculations for determining the new pixel locations. Let, $\Delta x = x_B - x_A$ and $\Delta y = y_B - y_A$ are the horizontal and vertical separations between the endpoint positions $A$ and $B$, and slope of the line $m = \Delta y / \Delta x$. Further, a decision parameter $P_k$, defined as

$$P_k = \Delta x.(d_1 - d_2) \qquad (5)$$

$$= \Delta x.(2.m.x_{k+1} - 2.y_k + 2.c - 1)$$

$$= 2.\Delta y.(x_k + 1) - 2.\Delta x.y_k + 2.c.\Delta x - \Delta x \qquad \text{(because } x_{k+1} = x_k + 1\text{)}$$

$$= 2.\Delta y.x_k - 2.\Delta x.y_k + \{2.\Delta y + \Delta x.(2c - 1)\}$$

or $$P_k = 2.\Delta y.x_k - 2.\Delta x.y_k + b \qquad (6)$$

where $b = 2.\Delta y + \Delta x.(2c - 1)$. For positive slope ($\Delta x > 0$), the sign of decision parameter $P_k$ will be same as the sign of $(d_1 - d_2)$, and parameter $b$ is independent of the pixel position. In fact, the parameter $b$ depends upon the incremental values ($\Delta x$ and $\Delta y$), which eliminates during the recursive calculations of decision parameter $P_k$. It is obvious from Fig. 2 that at new pixel position $x_{k+1}$, if pixel at $y_k$ position is closer to the actual line path (alternatively, $d_1 < d_2$) than the pixel at $y_{k+1}$ location, then decision parameter $P_k$ will be negative (eqn. 5) and lower pixel $y_k$ will be selected; otherwise, the upper pixel $y_{k+1}$ should be plotted. For recursive calculations, at step $k+1$, the decision parameter evaluated as

$$P_{k+1} = 2.\Delta y.x_{k+1} - 2.\Delta x.y_{k+1} + b \qquad (7)$$

Thus, $P_{k+1} - P_k = \{ 2.\Delta y.x_{k+1} - 2.\Delta x.y_{k+1} + b \} - \{ 2.\Delta y.x_k - 2.\Delta x.y_k + b \}$

$$= 2.\Delta y.(x_k + 1) - 2.\Delta x.y_{k+1} - 2.\Delta y.x_k + 2.\Delta x.y_k \qquad \text{(because } x_{k+1} = x_k + 1\text{)}$$

or $$P_{k+1} = P_k + 2.\Delta y - 2.\Delta x.(y_{k+1} - y_k) \qquad (8)$$

Depending upon sign of the decision parameter $P_k$, either $y_{k+1} - y_k = 0$ or 1. When parameter $P_k$ is negative (alternatively, $d_1 < d_2$), $y_{k+1} - y_k = 0$ or $y_{k+1} = y_k$; therefore, pixel at $y_k$ should be brightened. However, if $P_k$ is positive (alternatively, $d_1 > d_2$), $y_{k+1} - y_k = 1$ or $y_{k+1} = y_k + 1$; therefore, pixel $y_{k+1}$ becomes closer to the actual line path; hence, this should be energized.

The recursive calculations of decision parameter $P_k$ determine each integer $x$ pixel position, starting from left coordinate endpoint $(x_A, y_A)$ of the line $AB$. The initial value of decision parameter $P_0$ may be evaluated for the starting pixel position $(x_A, y_A)$, with the assumption that line passes through the origin (alternatively, $x_k = y_k = 0$ and $c = 0$, from eqn. 6) as

$$P_0 = 2.\Delta y.0 - 2.\Delta x.0 + \{2.\Delta y + \Delta x.(2.0 - 1)\} = 2.\Delta y - \Delta x \qquad (9)$$

**Algorithm:**

***Line generation algorithm when slope m <1 (i.e., slope is in between 0$^o$ and 45$^o$)***

***Step 1*:** Input the line endpoints $AB$ as $(x_A, y_A)$ and $(x_B, y_B)$, respectively.

***Step 2*:** Plot the first endpoint $(x_A, y_A)$, i.e., load the point into the frame buffer.

***Step 3*:** Calculate the initial value of decision parameter $P_k$, i.e. $P_0$

$$\Delta x = x_B - x_A \,;\, \Delta y = y_B - y_A \text{ and } P_0 = 2.\Delta y - \Delta x$$

***Step 4*:** At each $x_k$ position, starting from $k = 0$, perform the following test:

If $P_k < 1$ (i.e., negative), the next position to plot the pixel is $(x_k + 1, y_k)$ and $P_{k+1} = P_k + 2.\Delta y$

Otherwise ($P_k > 1$, i.e., positive), the next pixel position is $(x_k + 1, y_k + 1)$

and $P_{k+1} = P_k + 2.\Delta y - 2.\Delta x$

***Step 5*:** Plot a point of current $(x, y)$ position.

***Step 6*:** Repeat Step 4 through Step 5, $\Delta x$ times until reach the second endpoint, i.e. $x \geq x_B$.

***Line generation algorithm when slope m >1 (i.e., slope is in between 45$^o$ and 90$^o$)***

The procedure for line generation algorithm is same except that $x$ and $y$ coordinates are interchanged.

***Step 1*:** Input the line endpoints $AB$ as $(x_A, y_A)$ and $(x_B, y_B)$, respectively.

***Step 2*:** Plot the first endpoint $(x_A, y_A)$, i.e., load the point into the frame buffer.

***Step 3*:** Calculate the initial value of the decision parameter $P_k$, i.e. $P_0$

$$\Delta x = x_B - x_A \,;\, \Delta y = y_B - y_A \text{ and } P_0 = 2.\Delta x - \Delta y$$

***Step 4*:** At each $x_k$ position, starting from $k = 0$, perform the following test:

If $P_k < 1$ (i.e., negative), the next position to plot the pixel is $(x_k, y_k + 1)$ and $P_{k+1} = P_k + 2.\Delta x$

Otherwise ($P_k > 1$, i.e., positive), the next pixel position is $(x_k + 1, y_k + 1)$

and $P_{k+1} = P_k + 2.\Delta x - 2.\Delta y$

***Step 5*:** Plot a point of current $(x, y)$ position.

***Step 6*:** Repeat Step 4 through Step 5, $\Delta y$ times until reach the second endpoint, i.e. $y \geq y_B$.

**Program:**

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>
void main()
{
int x,y,x1,y1,x2,y2,p,dx,dy;
int gd=DETECT,gm;
initgraph(&gd,&gm,"C:\\TurboC3\\BGI");
printf("\nEnter the x-coordinate of the first point ::");
```

```
scanf("%d",&x1);
printf("\nEnter the y-coordinate of the first point ::");
scanf("%d",&y1);
printf("\nEnter the x-coordinate of the second point ::");
scanf("%d",&x2);
printf("\nEnter the y-coordinate of the second point ::");
scanf("%d",&y2);
x=x1;
y=y1;
dx=x2-x1;
dy=y2-y1;
putpixel(x,y,2);
p=(2*dy-dx);
while(x<=x2)
{
if(p<0)
{
x=x+1;
p=p+2*dy;
}
else
{
x=x+1;
y=y+1;
p=p+(2*dy)-(2*dx);
}
putpixel(x,y,7);
}
getch();
closegraph();
}
```